

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ASSIGNMENT 04
MÔN HỌC: PHÁT TRIỂN HỆ THỐNG THÔNG MINH

Lớp D23CTPM01

Giảng viên: Trần Đình Quế

Họ và Tên

Dương Trọng Thắng

Mã sinh viên

B23DCCN752

Hà Nội – 2026

Mục lục

Thông tin dự án	1
1 Bài toán, dữ liệu và quy trình đánh giá	2
1.1 Mục tiêu và phạm vi thực nghiệm	2
1.2 Hai bộ dữ liệu ở hai mức độ khó	2
1.3 Tách dữ liệu và chuẩn hóa	3
1.4 Metric và nguyên tắc đọc kết quả	3
2 Từ nền tảng CNN đến ba cách cài đặt	4
2.1 Phép tích chập và feature map	4
2.2 ReLU, pooling và receptive field	4
2.3 Softmax, Cross-Entropy và backpropagation	5
2.4 Training loop thuần NumPy	5
2.5 Cấu hình mô hình và chính sách huấn luyện	6
2.6 Vị trí của residual network trong tiến trình CNN	7
3 Đối chuẩn trên MNIST	8
3.1 Dữ liệu và cấu hình riêng	8
3.2 Kết quả test set	9
3.3 Diễn biến huấn luyện	10
3.4 Ma trận nhầm lẫn và phân tích lỗi	10
4 Đối chuẩn trên CIFAR-10	12
4.1 Dữ liệu và cấu hình riêng	12
4.2 Kết quả test set	13
4.3 Diễn biến huấn luyện	14
4.4 Độ khó theo lớp và các lỗi nổi bật	14
5 Đối chiếu tổng hợp và kết luận	16
5.1 Kết quả chính giữa hai dataset	16
5.2 Vai trò của ba cách cài đặt	16
5.3 Giới hạn của phép đối chuẩn	17
5.4 Khả năng tái lập	17
5.5 Kết luận và hướng mở rộng	18

Danh mục hình ảnh

3.1	Phân phối lớp của MNIST trên tập train gốc và test	8
3.2	Một ảnh đại diện cho mỗi lớp chữ số trong MNIST	8
3.3	Bốn metric phân loại của ba mô hình trên MNIST	9
3.4	Train/validation loss và accuracy trên MNIST	10
3.5	Ma trận nhầm lẫn chuẩn hóa của ba mô hình trên MNIST	10
3.6	Các lỗi có confidence cao nhất của PyTorch CNN trên MNIST	11
4.1	Phân phối cân bằng của CIFAR-10	12
4.2	Một ảnh đại diện cho mỗi lớp trong CIFAR-10	12
4.3	Bốn metric phân loại của ba mô hình trên CIFAR-10	13
4.4	Train/validation loss và accuracy trên CIFAR-10	14
4.5	Ma trận nhầm lẫn chuẩn hóa của ba mô hình trên CIFAR-10	14
4.6	Các lỗi có confidence cao nhất của PyTorch CNN trên CIFAR-10	15

Danh mục bảng biểu

1.1	Tổng quan dữ liệu dùng trong Assignment 04	2
1.2	Kích thước các tập dữ liệu thực tế trong hai notebook	3
2.1	Cấu hình CNN NumPy theo từng dataset	6
2.2	Kiến trúc và cấu hình hai framework	6
2.3	Một số hướng phát triển tiêu biểu từ CNN cơ bản	7
3.1	Benchmark ba cách cài đặt CNN trên MNIST	9
3.2	Accuracy theo lớp của mô hình tốt nhất trên MNIST (PyTorch CNN)	11
4.1	Benchmark ba cách cài đặt CNN trên CIFAR-10	13
4.2	Accuracy theo lớp của mô hình tốt nhất trên CIFAR-10 (PyTorch CNN)	15
5.1	Mô hình tốt nhất và CNN NumPy trên mỗi dataset	16
5.2	Ưu điểm, hạn chế và vai trò của ba cách cài đặt	16
5.3	Checklist tái lập Assignment 04	17

Thông tin dự án

GitHub project: <https://github.com/duongthang2710/PTHTTM>

Assignment 04 được tổ chức thành hai notebook độc lập cho MNIST và CIFAR-10. Mỗi notebook thực hiện cùng một quy trình: tải dữ liệu, khảo sát phân phối lớp, tách train/validation, chuẩn hóa, huấn luyện ba phiên bản CNN, đánh giá trên test set và phân tích lỗi. Hai tệp nguồn dùng để đối chiếu toàn bộ số liệu trong báo cáo là:

- `Assignment4/mnist/notebook/mnist_cnn_benchmark.ipynb`;
- `Assignment4/cifar10/notebook/cifar10_cnn_benchmark.ipynb`.

Dữ liệu được notebook tải hoặc đọc từ thư mục data cục bộ của từng dataset. Các hình đã xuất từ output thực nghiệm nằm trong `Assignment4/Report/images/generated`; báo cáo $\text{\LaTeX}$ nằm trong `Assignment4/Report`.

Bài toán, dữ liệu và quy trình đánh giá

1.1 Mục tiêu và phạm vi thực nghiệm

Assignment 04 khảo sát bài toán phân loại ảnh đa lớp bằng ba mức độ cài đặt:

1. **CNN thuần NumPy**: tự hiện thực convolution, max pooling, dense, softmax-cross-entropy, backpropagation và Adam, không dùng autograd;
2. **TensorFlow/Keras**: xây dựng mô hình bằng API cấp cao, automatic differentiation và pipeline `tf.data`;
3. **PyTorch**: định nghĩa module và training loop tường minh, đồng thời tận dụng backend tăng tốc của framework.

Mục tiêu không chỉ là tìm accuracy cao nhất. Thực nghiệm còn ghi nhận Macro-Precision, Macro-Recall, Macro-F1, số tham số, epoch tốt nhất, learning rate cuối, thời gian huấn luyện và thời gian dự đoán. Cách thiết kế này cho phép đối chiếu đồng thời chất lượng dự báo, độ phức tạp và mức độ thuận tiện của ba cách cài đặt.

1.2 Hai bộ dữ liệu ở hai mức độ khó

Bảng 1.1: Tổng quan dữ liệu dùng trong Assignment 04

Dataset	Train gốc	Test	Kích thước ảnh	Số lớp	Đặc điểm
MNIST	60,000	10,000	$28 \times 28 \times 1$	10	Chữ số viết tay, nền đơn giản
CIFAR-10	50,000	10,000	$32 \times 32 \times 3$	10	Vật thể tự nhiên, nền và góc nhìn đa dạng

MNIST gồm ảnh xám của mười chữ số viết tay. Đối tượng thường nằm gần trung tâm, nền tối và hình dạng tương đối rõ nên bộ dữ liệu phù hợp để kiểm tra tính đúng đắn của CNN tự cài đặt. CIFAR-10 gồm ảnh màu thuộc mười nhóm vật thể. Độ phân giải chỉ 32×32 trong khi nền, góc nhìn, màu sắc và hình dạng thay đổi mạnh, vì vậy mô hình cần học đặc trưng phức tạp hơn. Sự chênh lệch này giúp thể hiện rõ giới hạn của CNN nông và lợi ích của residual learning.

1.3 Tách dữ liệu và chuẩn hóa

Notebook dùng `RANDOM_STATE = 42` và tách train/validation theo nhãn bằng `stratify`. Hai framework sử dụng toàn bộ phần train còn lại sau khi giữ validation set; CNN NumPy dùng subset stratified để backpropagation thủ công vẫn có thời gian chạy khả thi. Tất cả mô hình của cùng dataset được đánh giá trên cùng full test set.

Bảng 1.2: Kích thước các tập dữ liệu thực tế trong hai notebook

Dataset	Framework train	Validation	Test	NumPy train	NumPy val
MNIST	54,000	6,000	10,000	20,000	3,000
CIFAR-10	45,000	5,000	10,000	15,000	3,000

Ảnh được đổi sang `float32`, chia cho 255 rồi chuẩn hóa theo từng channel. Mean và standard deviation chỉ được tính từ framework train set; cùng thống kê đó sau đó được áp dụng cho validation, test và subset NumPy:

$$\mathbf{X}_{norm} = \frac{\mathbf{X}/255 - \mu_{train}}{\sigma_{train} + 10^{-7}}. \quad (1.1)$$

Quy trình này tránh dùng thông tin từ validation hoặc test trong tiền xử lý. Test set cũng không tham gia chọn epoch, learning rate hay checkpoint.

1.4 Metric và nguyên tắc đọc kết quả

Accuracy đo tỉ lệ mẫu được phân loại đúng:

$$\text{Accuracy} = \frac{\text{số dự đoán đúng}}{N}. \quad (1.2)$$

Với từng lớp k , precision, recall và F1 được tính theo one-vs-rest. Macro average lấy trung bình không trọng số trên mười lớp, chẳng hạn:

$$\text{MacroF1} = \frac{1}{K} \sum_{k=1}^K F1_k. \quad (1.3)$$

Macro-F1 giúp mỗi lớp có ảnh hưởng ngang nhau, đặc biệt hữu ích khi phân phối MNIST không hoàn toàn đều. Trong notebook, bảng kết quả được sắp theo Macro-F1 rồi Accuracy. Ma trận nhầm lẫn chuẩn hóa và accuracy theo lớp được dùng để bổ sung cho các score tổng hợp; nhóm ảnh dự đoán sai có confidence cao nhất giúp kiểm tra kiểu lỗi cụ thể.

Thời gian huấn luyện và dự đoán chỉ phản ánh lần chạy đã lưu. NumPy dùng ít mẫu hơn, kiến trúc khác framework và các framework chịu ảnh hưởng của backend, compilation, warm-up cùng tải hệ thống. Vì vậy các số thời gian không được xem như benchmark phần cứng tuyệt đối.

Từ nền tảng CNN đến ba cách cài đặt

2.1 Phép tích chập và feature map

Ảnh số được biểu diễn bởi tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$. Một CNN ánh xạ ảnh sang logits $\mathbf{z} \in \mathbb{R}^K$, sau đó softmax chuyển logits thành phân phối xác suất. So với việc làm phẳng ảnh ngay từ đầu, convolution giữ quan hệ lân cận, dùng kết nối cục bộ và chia sẻ cùng kernel ở nhiều vị trí.

Với kernel $K_h \times K_w$, stride S và padding P , feature map thứ k có phần tử:

$$Y_{i,j,k} = b_k + \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} \sum_{c=0}^{C-1} X_{iS+u-P, jS+v-P, c} W_{u,v,c,k}. \quad (2.1)$$

Kích thước không gian đầu ra là:

$$H_{out} = \left\lfloor \frac{H + 2P - K_h}{S} \right\rfloor + 1, \quad (2.2)$$

$$W_{out} = \left\lfloor \frac{W + 2P - K_w}{S} \right\rfloor + 1. \quad (2.3)$$

Các convolution trong notebook dùng kernel 3×3 . Với padding 1 và stride 1, chiều cao và chiều rộng được giữ nguyên.

2.2 ReLU, pooling và receptive field

Sau convolution, ReLU tạo tính phi tuyến:

$$\text{ReLU}(z) = \max(0, z). \quad (2.4)$$

Max pooling 2×2 lấy kích hoạt lớn nhất trong từng vùng và giảm một nửa kích thước không gian. Việc xếp chồng nhiều convolution làm receptive field tăng dần, cho phép neuron ở tầng sâu tổng hợp thông tin từ vùng lớn hơn của ảnh.

CNN NumPy trong cả hai notebook có pipeline:

$$\text{Conv-ReLU-Pool} \rightarrow \text{Conv-ReLU-Pool} \rightarrow \text{Dense-ReLU} \rightarrow \text{Dense}. \quad (2.5)$$

Kiến trúc đủ ngắn để tự hiện thực toàn bộ forward/backward nhưng vẫn tạo hai cấp feature map.

2.3 Softmax, Cross-Entropy và backpropagation

Với logits $\mathbf{z}$, xác suất của lớp k là:

$$\hat{y}_k = \frac{\exp(z_k - \max_j z_j)}{\sum_{r=1}^K \exp(z_r - \max_j z_j)}. \quad (2.6)$$

Phép trừ logits lớn nhất trước khi lấy hàm mũ giúp tránh overflow. Loss trung bình trên mini-batch m mẫu được tính bởi:

$$\mathcal{L}_{CE} = -\frac{1}{m} \sum_{i=1}^m \log(\hat{y}_{i,y_i}). \quad (2.7)$$

Đúng với hàm `SoftmaxCrossEntropy.backward` trong notebook, gradient theo logits là:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_i} = \frac{\hat{\mathbf{y}}_i - \mathbf{y}_i}{m}. \quad (2.8)$$

Trong convolution, gradient trọng số được cộng dồn từ mọi vị trí sử dụng cùng kernel. CNN NumPy trích các cửa sổ bằng `sliding_window_view` và dùng `einsum` cho phép nhân tensor. Max pooling lưu mask cực đại; nếu một vùng có nhiều giá trị cực đại bằng nhau, gradient được chia đều cho các vị trí đó. Các lớp dense áp dụng:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{A}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{Z}}, \quad (2.9)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_i \frac{\partial \mathcal{L}}{\partial \mathbf{z}_i}, \quad (2.10)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}} \mathbf{W}^\top. \quad (2.11)$$

2.4 Training loop thuần NumPy

Mỗi epoch hoán vị train set, tăng cường từng mini-batch, thực hiện forward, backward và cập nhật Adam. Gradient của từng tensor được clip theo phần tử trong $[-5, 5]$; weight decay chỉ áp dụng cho ma trận trọng số. Learning rate giảm theo cosine từ 10^{-3} xuống 5×10^{-5} .

```

1 for epoch in range(epochs):
2     lr = cosine_learning_rate(initial_lr, epoch, epochs)
3     indices = rng.permutation(len(X_train))
4
5     for start in range(0, len(indices), batch_size):
6         batch_ids = indices[start:start + batch_size]
7         X_batch = augment_numpy_batch(X_train[batch_ids], rng, dataset_key)
8         y_batch = y_train[batch_ids]
9
10        logits = model.forward(X_batch)
11        loss = criterion.forward(logits, y_batch)
12        model.backward(criterion.backward())
13        optimizer.step(model.named_parameters(), learning_rate=lr)
14
15    if validation_loss < best_loss:
16        best_state = model.state_dict()
17

```

```
18 model.load_state_dict(best_state)
```

Đoạn mã 2.1: Luồng huấn luyện CNN NumPy trong notebook

Optimizer chỉ nhận gradient từ train loss. Cuối mỗi epoch, validation loss và validation accuracy được tính độc lập. CNN NumPy lưu checkpoint có validation loss thấp nhất; Keras và PyTorch lưu checkpoint có validation accuracy cao nhất. Trạng thái tốt nhất được khôi phục trước khi dự đoán test set.

2.5 Cấu hình mô hình và chính sách huấn luyện

Bảng 2.1: Cấu hình CNN NumPy theo từng dataset

Dataset	Filters	Hidden	Epoch	Batch	LR đầu	Weight decay	Tham số	Augmentation
MNIST	(8, 16)	64	15	64	10^{-3}	10^{-4}	52,138	Dịch tối đa 2 px
CIFAR-10	(12, 24)	128	50	64	10^{-3}	10^{-4}	200,978	Crop 4 px, flip

Bảng 2.2: Kiến trúc và cấu hình hai framework

Dataset	Kiến trúc	Optimizer	Epoch / Batch	Regularization
MNIST	2 block Conv-BN-ReLU-Pool, Dense 128	AdamW, LR 10^{-3}	15 / 128	Dropout 0.15, 0.25, 0.30
CIFAR-10	ResNet-20, stage 16 $\rightarrow$ 32 $\rightarrow$ 64	SGD, momentum 0.9, Nesterov	50 / 256	$L2 \ 5 \times 10^{-4}$, smoothing 0.1

Trên MNIST, Keras và PyTorch có cùng topology trainable: mỗi block chứa hai convolution 3×3 , Batch Normalization và ReLU, tiếp theo là max pooling và Dropout; phần phân loại dùng Dense 128 rồi output 10 lớp. Hai framework dùng AdamW và giảm learning rate khi validation accuracy ngừng cải thiện.

Trên CIFAR-10, hai framework dùng ResNet-20: stem 16 channel, ba stage 16 $\rightarrow$ 32 $\rightarrow$ 64, mỗi stage có ba residual block, sau cùng là Global Average Pooling và lớp tuyến tính 10 đầu ra. Khi kích thước hoặc số channel thay đổi, nhánh tắt dùng convolution 1×1 với stride tương ứng. Cả hai dùng SGD, momentum 0.9, Nesterov, warm-up 5 epoch từ 0.02 lên 0.10 và cosine decay; Cross-Entropy có label smoothing 0.1.

Augmentation chỉ áp dụng cho train. MNIST dùng reflect padding/crop hoặc phép tịnh tiến tương đương với độ dịch tối đa 2 pixel. CIFAR-10 dùng dịch tối đa 4 pixel và horizontal flip với xác suất 0.5. Validation và test không augmentation.

2.6 Vị trí của residual network trong tiến trình CNN

Bảng 2.3: Một số hướng phát triển tiêu biểu từ CNN cơ bản

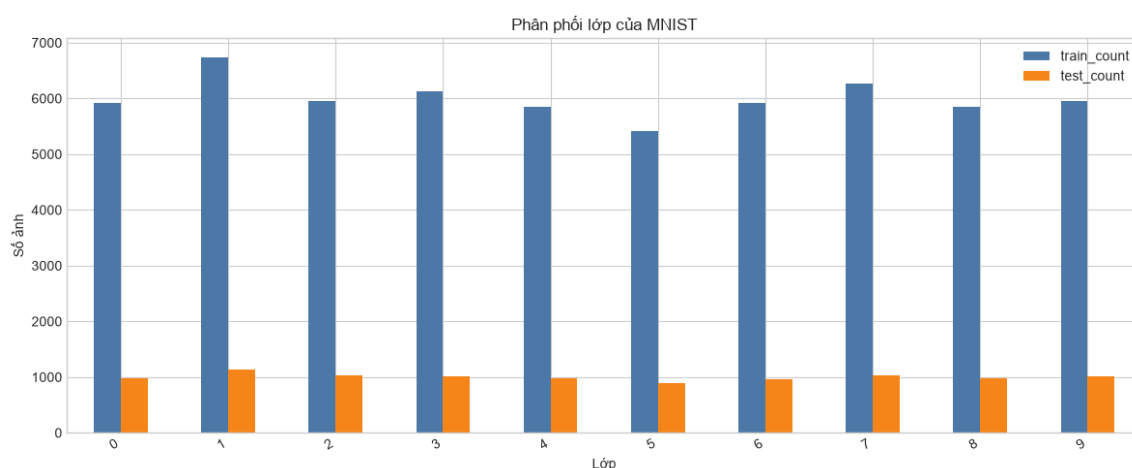
Kiến trúc	Cải tiến chính	Ý nghĩa
LeNet-5	Conv, pooling và fully connected theo chuỗi	Nền tảng cho nhận dạng chữ số
AlexNet	ReLU, dropout, augmentation, GPU	Mở rộng CNN sâu lên ImageNet
VGG	Xếp chồng convolution 3×3	Thiết kế đồng nhất nhưng nhiều tham số
Inception	Nhiều nhánh kernel và convolution 1×1	Học đặc trưng đa tỉ lệ
ResNet	Skip/residual connection	Cải thiện luồng gradient trong mạng sâu
DenseNet	Nối feature từ các tầng trước	Tái sử dụng feature
MobileNet	Depthwise-separable convolution	Giảm FLOPs và số tham số
EfficientNet	Scale depth, width và resolution	Cân bằng chất lượng với chi phí

Residual block học $F(\mathbf{x})$ rồi cộng lại đầu vào, $\mathbf{y} = F(\mathbf{x}; \theta) + \mathbf{x}$. Đường truyền tắt giúp gradient đi qua mạng sâu thuận lợi hơn. Đây là lý do ResNet-20 được chọn cho CIFAR-10, trong khi mô hình nông vẫn đủ phù hợp với MNIST.

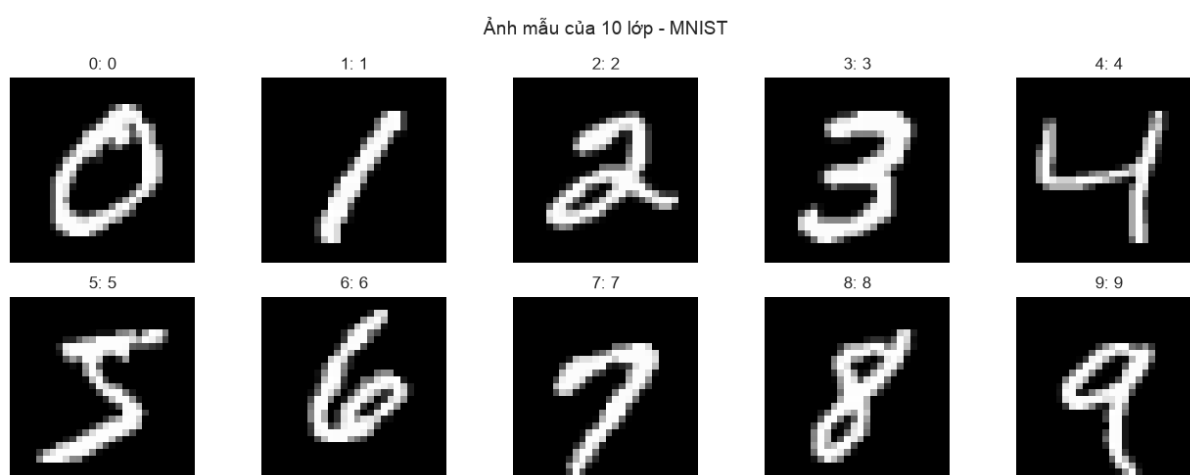
Đôi chuẩn trên MNIST

3.1 Dữ liệu và cấu hình riêng

MNIST có 60,000 ảnh train gốc và 10,000 ảnh test; pixel nằm trong khoảng 0 đến 255. Phân phối lớp không mất cân bằng nghiêm trọng nhưng không hoàn toàn đều: lớp 1 có 6,742 ảnh train, còn lớp 5 có 5,421 ảnh.



Hình 3.1: Phân phối lớp của MNIST trên tập train gốc và test



Hình 3.2: Một ảnh đại diện cho mỗi lớp chữ số trong MNIST

Framework train có 54,000 mẫu, validation có 6,000 mẫu; subset NumPy gồm 20,000 mẫu train và 3,000 mẫu validation. Thống kê chỉ học từ framework train là $\mu = 0.13073687$ và $\sigma = 0.30819046$.

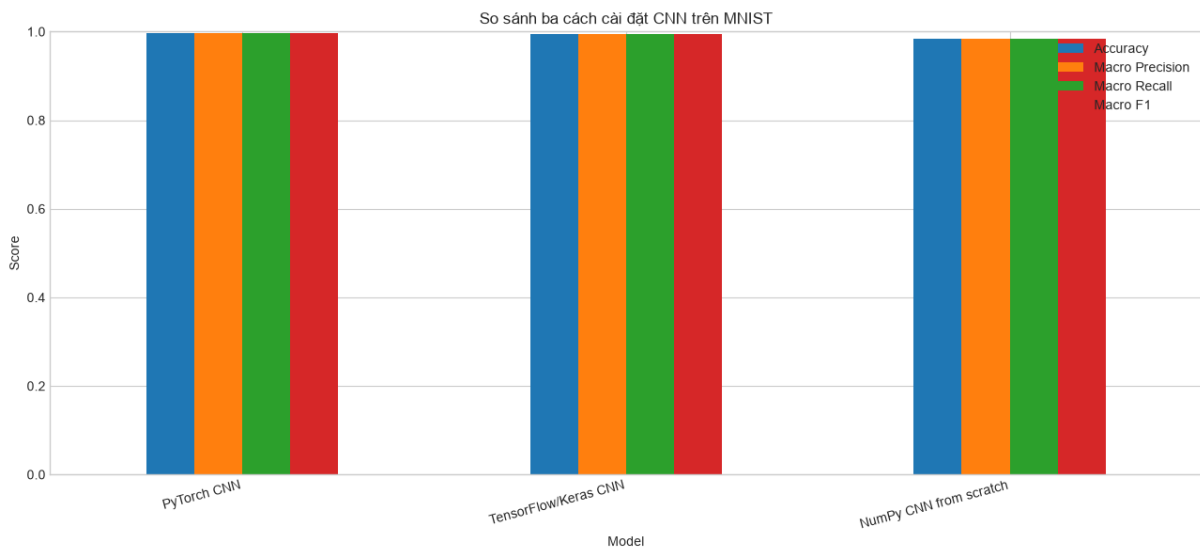
CNN NumPy có luồng kích thước $28 \times 28 \times 1 \rightarrow \text{Conv}(8) \rightarrow \text{Pool} \rightarrow \text{Conv}(16) \rightarrow \text{Pool} \rightarrow \text{Dense}(64) \rightarrow 10$, với 52,138 tham số. Keras báo cáo 468,394 tham số tổng, gồm 468,010 tham số trainable và 384 trạng thái không trainable của Batch Normalization. PyTorch có 468,010 tham số.

3.2 Kết quả test set

Bảng 3.1: Benchmark ba cách cài đặt CNN trên MNIST

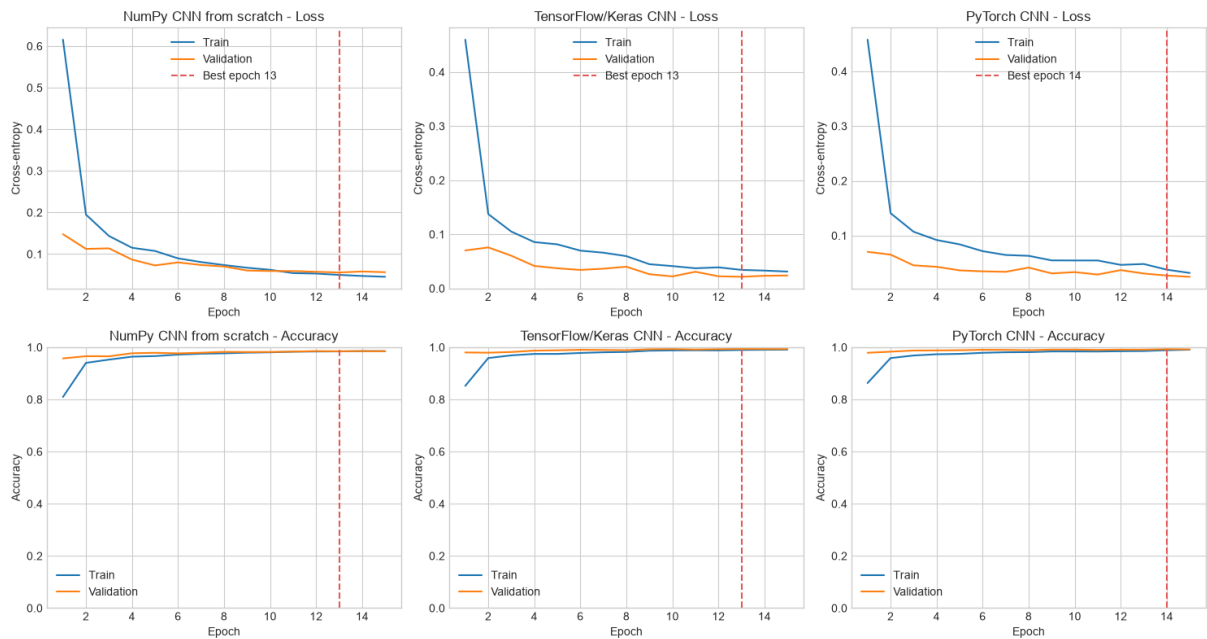
Mô hình	Acc.	Macro-P	Macro-R	Macro-F1	Train	Epoch	LR cuối	Tham số	Train(s)	Pred(s)
PyTorch CNN	0.9958	0.9959	0.9958	0.9958	54,000	14	3.0×10^{-4}	468,010	90.38	0.307
TensorFlow/Keras CNN	0.9955	0.9955	0.9955	0.9955	54,000	13	2.7×10^{-5}	468,394	399.87	1.183
NumPy CNN from scratch	0.9848	0.9847	0.9847	0.9847	20,000	13	5.0×10^{-5}	52,138	82.86	1.094

PyTorch có Macro-F1 cao nhất 0.9958, hơn Keras 0.0003; chênh lệch về chất lượng dự báo là rất nhỏ. Trong lần chạy lưu trong notebook, PyTorch dự đoán nhanh nhất và train trong 90.38 giây trên MPS. CNN NumPy đạt accuracy 0.9848 dù chỉ dùng 20,000 mẫu train và 52,138 tham số, cho thấy forward/backward thủ công hoạt động đúng. Khoảng cách xấp xỉ 1.1 điểm phần trăm so với framework còn chịu ảnh hưởng của lượng dữ liệu, độ lớn mô hình và primitive tối ưu mức thấp.



Hình 3.3: Bốn metric phân loại của ba mô hình trên MNIST

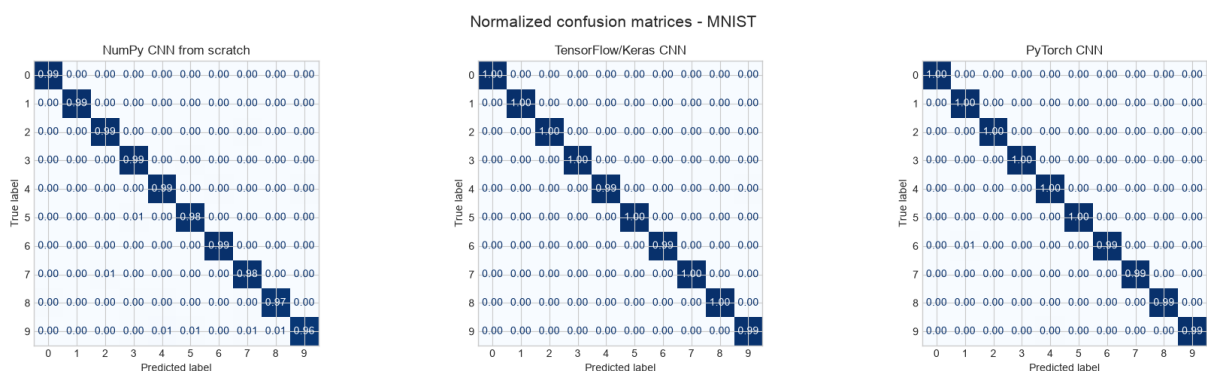
3.3 Diễn biến huấn luyện



Hình 3.4: Train/validation loss và accuracy trên MNIST

CNN NumPy giảm validation loss từ 0.1477 ở epoch 1 xuống 0.0558 ở epoch 13. Hai epoch sau có validation loss lần lượt 0.0581 và 0.0564, nên notebook khôi phục checkpoint epoch 13. Keras đạt validation accuracy tốt nhất 0.9937 ở epoch 13; PyTorch đạt 0.9928 ở epoch 14. Các đường train/validation của framework hội tụ trên 0.99 và không tách xa, cho thấy augmentation cùng regularization kiểm soát overfitting tốt trong cấu hình này.

3.4 Ma trận nhầm lẫn và phân tích lỗi



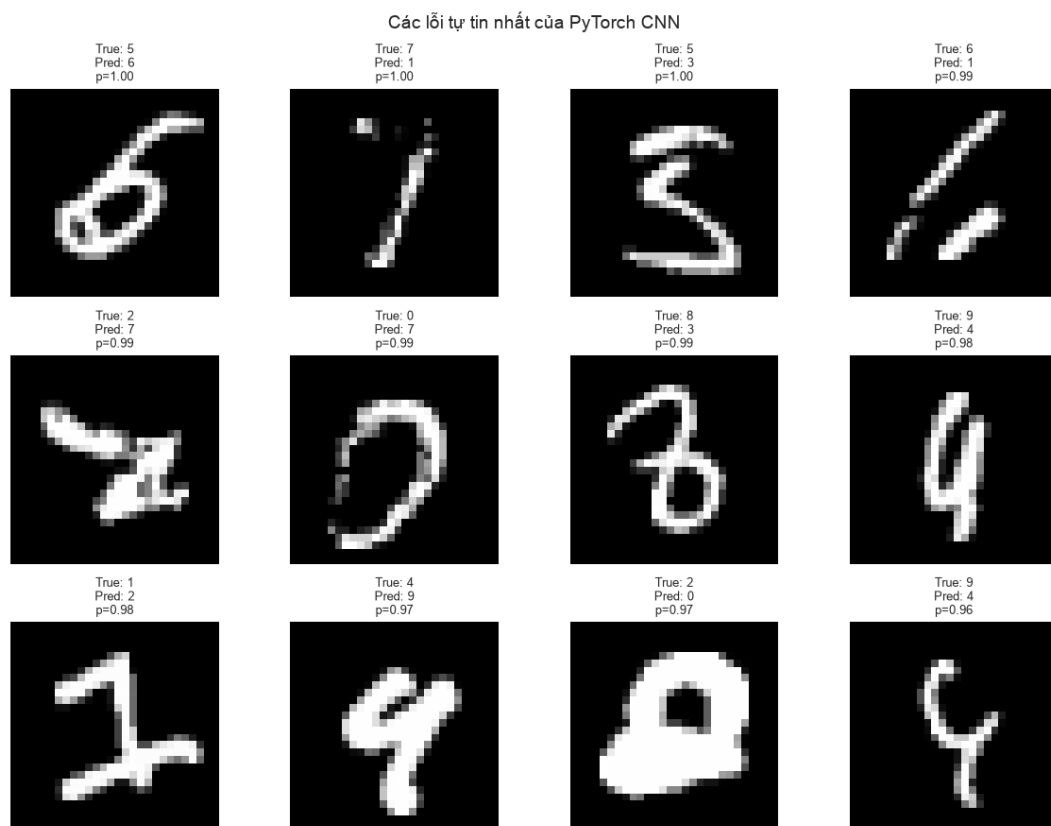
Hình 3.5: Ma trận nhầm lẫn chuẩn hóa của ba mô hình trên MNIST

Hai framework có ma trận gần như tập trung hoàn toàn trên đường chéo. CNN NumPy yếu hơn ở chữ số 8 và 9, với recall hiển thị xấp xỉ 0.97 và 0.96. Các chữ số có nét vòng hoặc nét kéo dài dễ bị nhầm khi cách viết không điển hình.

Bảng 3.2: Accuracy theo lớp của mô hình tốt nhất trên MNIST (PyTorch CNN)

Lớp	Số mẫu	Accuracy	Lớp	Số mẫu	Accuracy
0	980	0.9990	5	892	0.9955
1	1,135	0.9974	6	958	0.9927
2	1,032	0.9971	7	1,028	0.9942
3	1,010	0.9960	8	974	0.9949
4	982	0.9990	9	1,009	0.9921

Lớp 0 và 4 cùng đạt 0.9990; lớp 9 thấp nhất nhưng vẫn đạt 0.9921. Notebook chọn 12 dự đoán sai có confidence lớn nhất của mô hình đứng đầu theo Macro-F1.



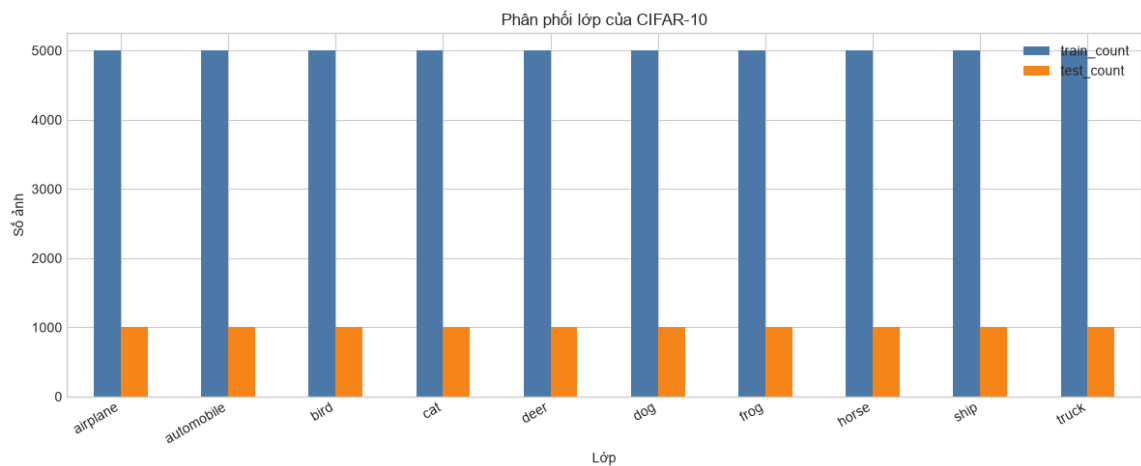
Hình 3.6: Các lỗi có confidence cao nhất của PyTorch CNN trên MNIST

Nhiều ảnh sai có nét viết mơ hồ: chữ số 5 gần giống 6 hoặc 3, chữ số 7 giống 1, và chữ số 9 gần giống 4. Kết quả nhắc lại rằng confidence softmax cao không bảo đảm dự đoán đúng khi hình dạng nằm ngoài kiểu phổ biến của train set.

Đôi chuẩn trên CIFAR-10

4.1 Dữ liệu và cấu hình riêng

CIFAR-10 có 50,000 ảnh train và 10,000 ảnh test thuộc các lớp *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship*, *truck*. Mỗi lớp có đúng 5,000 ảnh train gốc và 1,000 ảnh test; vì test set cân bằng, Accuracy và Macro-Recall bằng nhau trong bảng kết quả.



Hình 4.1: Phân phối cân bằng của CIFAR-10



Hình 4.2: Một ảnh đại diện cho mỗi lớp trong CIFAR-10

Framework train gồm 45,000 mẫu, validation gồm 5,000 mẫu. Subset NumPy có 15,000 mẫu train và 3,000 mẫu validation; riêng train subset có đúng 1,500 mẫu mỗi lớp. Mean và standard deviation học từ framework train là:

$$\mu_{train} = (0.49113446, 0.48211210, 0.44644734), \quad (4.1)$$

$$\sigma_{train} = (0.24686947, 0.24343061, 0.26160370). \quad (4.2)$$

CNN NumPy dùng 12 và 24 filter, hidden layer 128, tổng 200,978 tham số. Keras ResNet-20 có 274,042 tham số tổng, gồm 272,474 trainable và 1,568 trạng thái Batch Normalization; PyTorch ResNet-20 có 272,474 tham số.

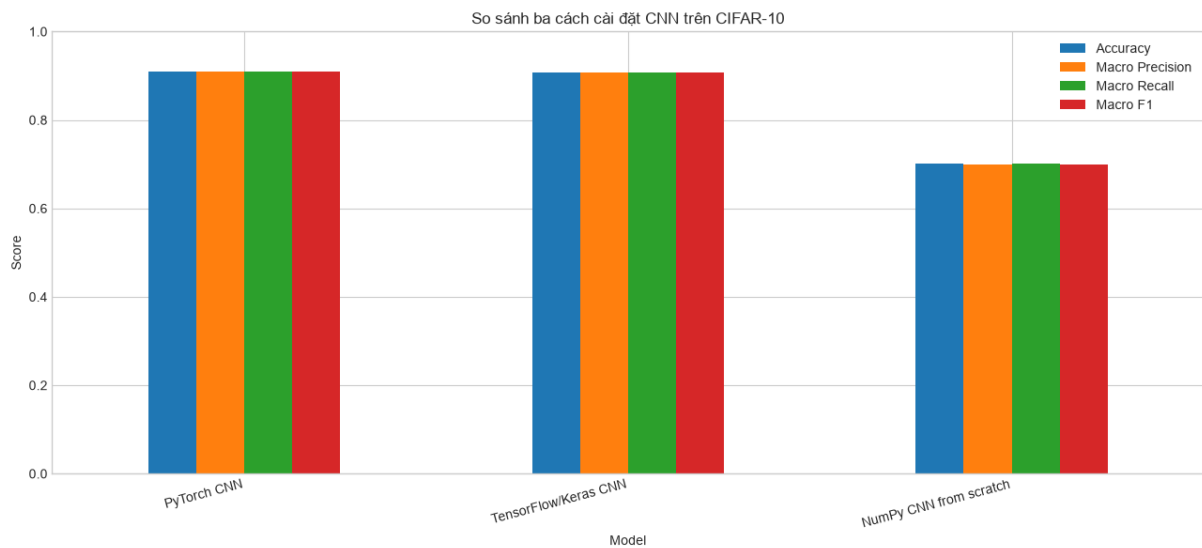
4.2 Kết quả test set

Bảng 4.1: Benchmark ba cách cài đặt CNN trên CIFAR-10

Mô hình	Acc.	Macro-P	Macro-R	Macro-F1	Train	Epoch	LR cuối	Tham số	Train(s)	Pred(s)
PyTorch CNN	0.9095	0.9092	0.9095	0.9092	45,000	48	2.22×10^{-4}	272,474	822.82	1.248
TensorFlow/Keras CNN	0.9078	0.9077	0.9078	0.9075	45,000	49	2.22×10^{-4}	274,042	2,636.45	4.125
NumPy CNN from scratch	0.7012	0.6987	0.7012	0.6994	15,000	47	5.0×10^{-5}	200,978	501.36	3.875

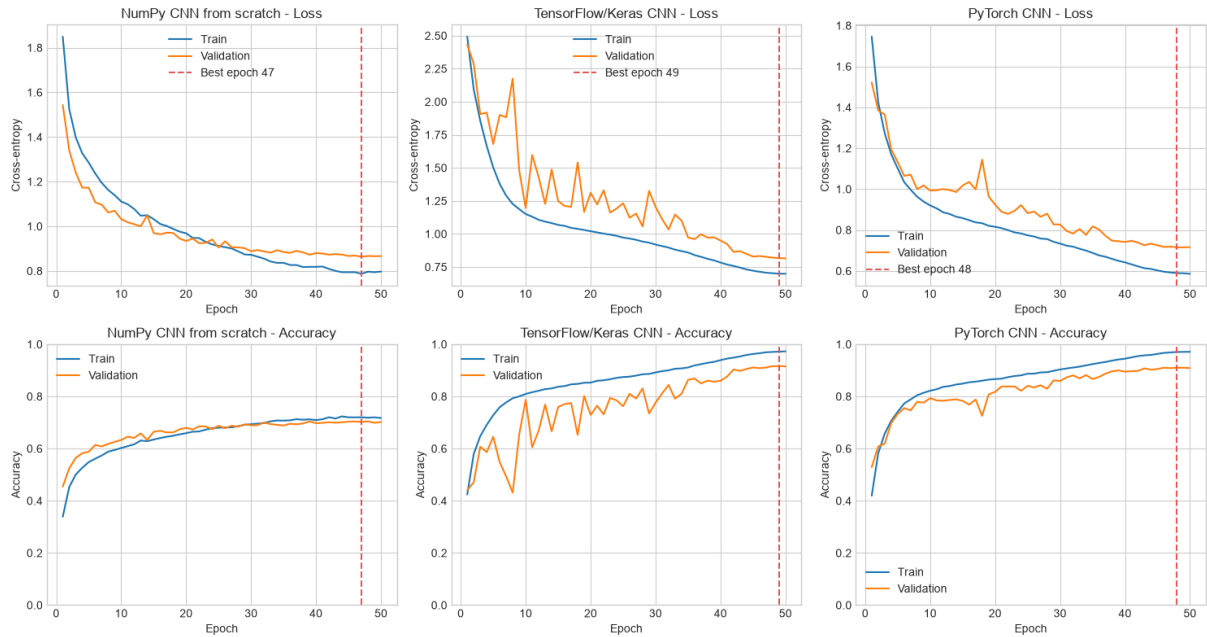
PyTorch có Macro-F1 cao nhất 0.9092, hơn Keras 0.0017. Hai ResNet-20 gần tương đương về chất lượng, nhưng thời gian của lần chạy đã lưu khác đáng kể: 822.82 giây so với 2,636.45 giây khi train và 1.248 giây so với 4.125 giây khi dự đoán. Đây là kết quả theo backend MPS và môi trường cụ thể, không phải kết luận rằng một framework luôn nhanh hơn framework còn lại.

CNN NumPy đạt accuracy 0.7012 và Macro-F1 0.6994. So với ResNet-20, khoảng cách lớn phản ánh đồng thời kiến trúc nông, subset chỉ 15,000 ảnh, ít tham số hơn và không có Batch Normalization hay residual connection.



Hình 4.3: Bốn metric phân loại của ba mô hình trên CIFAR-10

4.3 Diễn biến huấn luyện

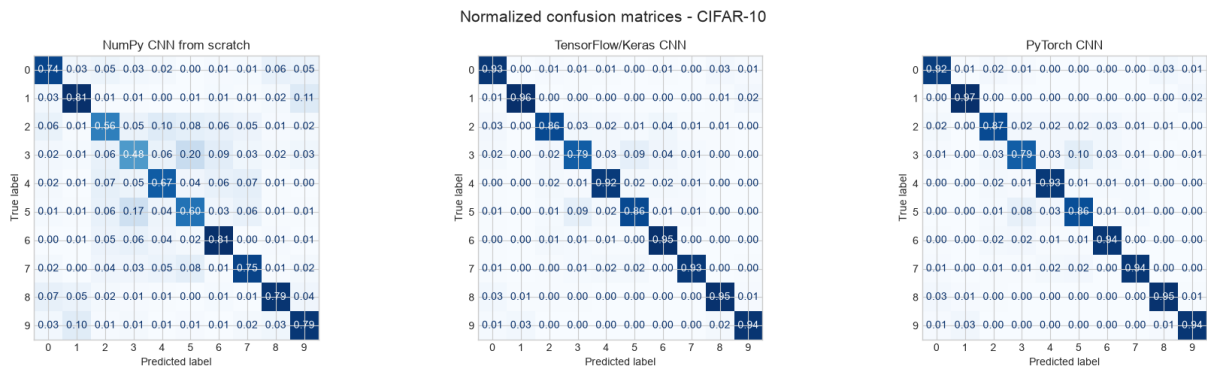


Hình 4.4: Train/validation loss và accuracy trên CIFAR-10

CNN NumPy dừng quanh train/validation accuracy 0.72/0.70; validation loss tốt nhất là 0.8634 tại epoch 47. Hai đường gần nhau cho thấy hạn chế chính là underfitting hơn là overfitting.

Keras đạt validation accuracy tốt nhất 0.9162 ở epoch 49; PyTorch đạt 0.9102 ở epoch 48. Keras có validation score cao hơn nhưng PyTorch nhìn hơn trên test, cho thấy chênh lệch nhỏ có thể thay đổi theo split và trạng thái huấn luyện. Validation metric dao động trong nửa đầu do augmentation, SGD và learning rate lớn; khi cosine decay hạ learning rate về cuối, các đường cong ổn định hơn.

4.4 Độ khó theo lớp và các lỗi nổi bật



Hình 4.5: Ma trận nhầm lẫn chuẩn hóa của ba mô hình trên CIFAR-10

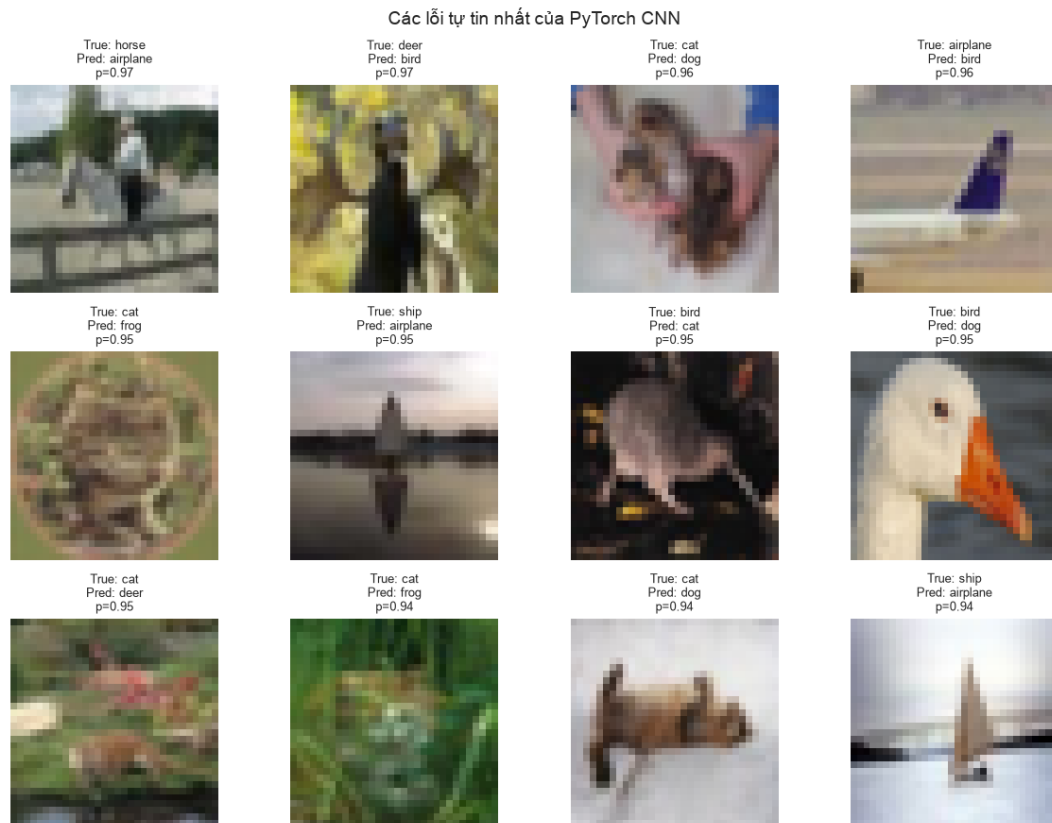
CNN NumPy nhầm nhiều giữa các lớp động vật. Trong ma trận làm tròn, recall lớp *cat* chỉ khoảng 0.48 và khoảng 20% ảnh mèo bị dự đoán thành chó. Lớp *bird* cũng dễ nhầm sang *deer*,

dog hoặc *frog*. Các lớp phương tiện có đường nét và bối cảnh đặc trưng hơn nên nhìn chung dễ phân loại hơn. ResNet-20 giảm mạnh các ô ngoài đường chéo, nhưng cặp *cat-dog* vẫn là nguồn lỗi đáng kể.

Bảng 4.2: Accuracy theo lớp của mô hình tốt nhất trên CIFAR-10 (PyTorch CNN)

Lớp	Số mẫu	Accuracy	Lớp	Số mẫu	Accuracy
airplane	1,000	0.9150	dog	1,000	0.8560
automobile	1,000	0.9660	frog	1,000	0.9390
bird	1,000	0.8660	horse	1,000	0.9360
cat	1,000	0.7940	ship	1,000	0.9510
deer	1,000	0.9290	truck	1,000	0.9430

Automobile tốt nhất với accuracy 0.9660, còn *cat* thấp nhất với 0.7940. Khoảng cách 17.2 điểm phần trăm cho thấy accuracy tổng 0.9095 không phản ánh đầy đủ khác biệt giữa các lớp.



Hình 4.6: Các lỗi có confidence cao nhất của PyTorch CNN trên CIFAR-10

Nhiều lỗi có tính mơ hồ ở độ phân giải 32×32 : ngựa bị che khuất có thể giống máy bay, hươu trong nền rừng giống chim, mèo/chó/chim bị nhầm do texture, hoặc thuyền và máy bay cùng xuất hiện trên nền trời/nước. Trong ứng dụng thực tế, có thể bổ sung calibration, dự đoán top- k hoặc cơ chế từ chối khi confidence không đáng tin cậy.

Đối chiếu tổng hợp và kết luận

5.1 Kết quả chính giữa hai dataset

Bảng 5.1: Mô hình tốt nhất và CNN NumPy trên mỗi dataset

Dataset	Mô hình	Accuracy	Macro-F1	Tham số	Train(s)	Dữ liệu train
MNIST	PyTorch CNN	0.9958	0.9958	468,010	90.38	54,000
MNIST	NumPy CNN	0.9848	0.9847	52,138	82.86	20,000
CIFAR-10	PyTorch ResNet-20	0.9095	0.9092	272,474	822.82	45,000
CIFAR-10	NumPy CNN	0.7012	0.6994	200,978	501.36	15,000

Trên MNIST, CNN nông đã đủ mạnh: phiên bản NumPy chỉ kém mô hình tốt nhất khoảng 1.1 điểm phần trăm accuracy. Trên CIFAR-10, cùng thiết kế nông dừng ở khoảng 70%, trong khi ResNet-20 vượt 90%. So sánh này minh họa rằng độ phức tạp kiến trúc cần phù hợp với độ phức tạp của dữ liệu.

5.2 Vai trò của ba cách cài đặt

Bảng 5.2: Ưu điểm, hạn chế và vai trò của ba cách cài đặt

Cách cài đặt	Ưu điểm	Hạn chế	Vai trò trong Assignment 04
NumPy from scratch	Minh bạch forward/backward, ít phụ thuộc	Chậm, khó mở rộng, dễ sai gradient	Hiểu thuật toán ở mức tensor
TensorFlow/Keras	API gọn, callback và pipeline dữ liệu tốt	Training loop ít tường minh hơn	Xây dựng baseline framework nhanh
PyTorch	Module và loop linh hoạt, debug thuận tiện	Phải tự quản lý nhiều trạng thái	Benchmark hiệu quả và kiểm soát rõ

CNN NumPy không nhằm thay thế framework trong triển khai quy mô lớn. Giá trị chính của nó là làm rõ cache, mask pooling, sliding window, gradient và cập nhật Adam. Keras và PyTorch

phù hợp hơn khi cần residual blocks, tăng tốc phần cứng, mixed precision hoặc mở rộng sang dữ liệu lớn.

5.3 Giới hạn của phép đối chuẩn

Các mô hình dùng cùng validation/test set và cùng metric trong từng dataset, nhưng điều kiện huấn luyện không hoàn toàn giống nhau:

1. NumPy dùng ít mẫu train hơn để giới hạn thời gian chạy;
2. kiến trúc NumPy nông hơn, còn framework dùng ResNet-20 trên CIFAR-10;
3. Keras `count_params()` gồm cả trạng thái Batch Normalization không trainable, trong khi phép đếm PyTorch cộng các parameter của mô hình;
4. thời gian phụ thuộc MPS, backend, graph compilation, warm-up và tải hệ thống tại thời điểm chạy;
5. báo cáo chỉ có một random seed, chưa có trung bình và độ lệch chuẩn qua nhiều lần lặp.

Vì vậy bảng kết quả trả lời câu hỏi “pipeline nào hiệu quả trong đúng cấu hình notebook này” hơn là chứng minh framework nào luôn nhanh hoặc chính xác hơn.

5.4 Khả năng tái lập

Bảng 5.3: Checklist tái lập Assignment 04

Hạng mục	Cách thực hiện trong code	Ý nghĩa
Random seed	<code>RANDOM_STATE = 42</code> cho NumPy/Keras/Torch	Giảm dao động split và khởi tạo
Tách dữ liệu	Stratified train/validation, test độc lập	Không chọn mô hình trên test
Chuẩn hóa	Mean/std chỉ học từ framework train	Chống data leakage
Checkpoint	NumPy theo val loss, framework theo val accuracy	Khôi phục trạng thái tốt nhất
Augmentation	Chỉ áp dụng cho mini-batch train	Giữ validation/test nguyên bản
Metric	Accuracy và macro P/R/F1 trên cùng test set	So sánh đa chiều
Hình ảnh	Xuất từ output notebook vào thư mục báo cáo	Truy vết được bảng và đồ thị

Output đã lưu ghi nhận NumPy 2.5.2, TensorFlow 2.21.0 và PyTorch 2.14.0; thiết bị PyTorch là MPS. Để chạy lại đúng cấu trúc hiện tại:

1. mở notebook trong Assignment4/<dataset>/notebook;
2. bảo đảm thư mục `../data` tồn tại để loader tải hoặc đọc dữ liệu cục bộ;
3. chạy toàn bộ cell theo thứ tự từ cấu hình, load/split/normalize đến NumPy, Keras, PyTorch, benchmark và error analysis;
4. xuất các đồ thị cần dùng vào Assignment4/Report/images/generated;
5. biên dịch A04_CT_ThangDT.752.tex bằng XeLaTeX hai lần để cập nhật mục lục và tham chiếu chéo.

5.5 Kết luận và hướng mở rộng

Assignment 04 đã hiện thực và đối chuẩn ba cách xây dựng CNN trên MNIST và CIFAR-10. CNN thuần NumPy bao gồm forward, backward, Adam, weight decay, gradient clipping, cosine learning rate và augmentation. Trên MNIST, PyTorch đạt Accuracy/Macro-F1 0.9958, Keras đạt 0.9955, còn NumPy đạt Accuracy 0.9848 và Macro-F1 0.9847. Trên CIFAR-10, PyTorch ResNet-20 đạt Accuracy 0.9095 và Macro-F1 0.9092; Keras gần tương đương, trong khi CNN NumPy nông đạt Macro-F1 0.6994.

Kết quả cho thấy residual connection, Batch Normalization, augmentation, label smoothing và lịch learning rate dài đặc biệt quan trọng đối với ảnh tự nhiên. Phân tích lỗi cũng cho thấy các lớp gần nhau về hình dạng hoặc texture vẫn dễ bị nhầm, rõ nhất là *cat* và *dog* trên CIFAR-10.

Hướng mở rộng gồm chạy nhiều seed để báo cáo độ ổn định, kiểm tra calibration, thử mixup/cutout, dùng transfer learning và đo thêm throughput, bộ nhớ cùng kích thước checkpoint. Với CNN NumPy, có thể tiếp tục vector hóa convolution hoặc sử dụng Numba/CuPy để tách ảnh hưởng của thuật toán khỏi overhead vòng lặp Python.